

Análise de Bugs na Lógica de Finalização de Tarefas

Data: 10 de Março de 2026

Branch: ModularizacaoSistema

Autor: Análise automatizada

Resumo Executivo

A análise identificou **5 bugs críticos** na lógica de finalização de tarefas, sendo o principal problema a **duplicação de funções** entre os serviços modularizados e o `taskExecutionService.js`. O código re-fatorado criou versões modularizadas das funções em serviços separados (`contractVerificationService.js`, `evidenceService.js`), porém o `taskExecutionService.js` **ainda contém cópias locais de-satualizadas** dessas funções e está usando-as ao invés dos serviços modularizados.

Bug 1: Duplicação de `verifyContract` (CRÍTICO)

Localização

- **Versão desatualizada (sendo usada):** `src/services/taskExecutionService.js`, linhas 444-458
- **Versão atualizada (ignorada):** `src/services/contractVerificationService.js`, linhas 19-290

Problema

O método `executeTask` na linha 485 chama `this.verifyContract()` que aponta para a versão **local simplificada**, ignorando completamente o `ContractVerificationService` que foi criado durante a modularização.

Código Problemático (taskExecutionService.js:444-458)

```
static async verifyContract(doneFile, relatorioFile, terminalLogFile, options = {}) {
  try {
    const doneExists = await this.fileExists(doneFile);
    const relatorioExists = await this.fileExists(relatorioFile);
    let executionNotes = '';
    if (relatorioExists) {
      const content = await fs.readFile(relatorioFile, 'utf8');
      if (content.trim().length > 10) executionNotes = content.trim();
    }
    const { taskType = 'automation', evidence = {} } = options;
    const filesModified = (evidence.modifiedFiles || []).filter(f => f !== doneFile
    && f !== relatorioFile);
    if (taskType === 'development' && filesModified.length === 0 && !doneExists) re-
    turn { contractFulfilled: false, feedbackToAgent: 'Nenhuma alteração real de-
    tectada.' };
    if (doneExists) return { contractFulfilled: true, executionNotes: executionNotes |
    'Concluído.' };
    return { contractFulfilled: false, executionNotes: 'Aguardando .done' };
  } catch (e) { return { contractFulfilled: false, executionNotes: e.message }; }
}
```

Impacto

A versão simplificada **NÃO verifica:**

- Relatório válido (mínimo 20 caracteres vs 10)
- Camadas obrigatórias (frontend/backend)
- Evidência de inspeção para tarefas de análise
- Verificações específicas por tipo de tarefa
- Feedback detalhado para o agente

Chamada Problemática (linha 485)

```
contractResult = await this.verifyContract(files.doneFile, files.relatorioFile,
files.terminalLogFile, { taskType: analysisPlan.taskType, evidence });
```

Sugestão de Correção

```
// Importar no topo do arquivo
const ContractVerificationService = require('./contractVerificationService');

// Substituir a chamada na linha 485 por:
contractResult = await ContractVerificationService.verifyContract(
  files.doneFile,
  files.relatorioFile,
  files.terminalLogFile,
  {
    taskType: analysisPlan.taskType,
    task,
    evidence,
    project,
    analysisPlan
  }
);
```

Bug 2: Duplicação de `computeTurnProgress` (CRÍTICO)

Localização

- **Versão desatualizada (sendo usada):** `src/services/taskExecutionService.js`, linhas 551-563
- **Versão atualizada (ignorada):** `src/services/evidenceService.js`, linhas 156-204

Problema

A função `computeTurnProgress` local **NÃO considera**:

1. `filesWritten` para detectar mutações
2. `touchedFiles` para detectar atividade de análise

Código Problemático (`taskExecutionService.js:551-563`)

```
function computeTurnProgress(toolResult = {}, contractResult = {}, cwd = process.cwd()) {
  let inferredReads = []; let inferredMutations = [];
  const cmds = Array.isArray(toolResult.commandsExecuted) ? toolResult.commandsExecuted : [];
  for (const c of cmds) {
    const inf = parseCommandEvidence(c, cwd);
    inferredReads = mergeUniquePaths(inferredReads, inf.filesRead);
    inferredMutations = mergeUniquePaths(inferredMutations, inf.modifiedFiles);
  }
  const hasReads = mergeUniquePaths(toolResult.filesRead || [], inferredReads).filter(f => !isEphemeralArtifact(f)).length > 0;
  const hasMutations = mergeUniquePaths(toolResult.modifiedFiles || [], inferredMutations).filter(f => !isEphemeralArtifact(f)).length > 0;
  const success = !!contractResult.contractFulfilled;
  return { hasMeaningfulProgress: success || hasReads || hasMutations };
}
```

Versão Correta (`evidenceService.js:156-204`)

```
static computeTurnProgress(toolResult = {}, contractResult = {}, cwd = process.cwd()) {
  // ... código completo ...
  // BUG FIX: Considerar TANTO modifiedFiles QUANTO filesWritten
  const allMutations = mergeUniquePaths(
    toolResult.modifiedFiles || [],
    toolResult.filesWritten || [] // <-- FALTA na versão local
  );
  // BUG FIX: Considerar touchedFiles para detectar atividade de análise
  const touchedFiles = (toolResult.touchedFiles || []).filter(...); // <-- FALTA na versão local
  // ...
}
```

Impacto

Tarefas de análise que leem arquivos mas não os modificam podem ser erroneamente marcadas como “sem progresso”, causando falso-negativos.

Chamada Problemática (linha 498)

```
const prog = computeTurnProgress(res.toolResult || {}, contractResult, project?.pastaBase || TASKS_DIR);
```

Sugestão de Correção

```
// Importar no topo do arquivo
const EvidenceService = require('./evidenceService');

// Substituir a chamada na linha 498 por:
const prog = EvidenceService.computeTurnProgress(res.toolResult || {}, contractResult, project?.pastaBase || TASKS_DIR);
```

Bug 3: Duplicação de isEphemeralArtifact (MODERADO)

Localização

- **Versão desatualizada (sendo usada):** `src/services/taskExecutionService.js`, linhas 515-519
- **Versão atualizada (ignorada):** `src/services/evidenceService.js`, linhas 105-142

Problema

A versão local tem regex diferente e menos padrões de arquivos efêmeros.

Código Problemático (taskExecutionService.js:515-519)

```
function isEphemeralArtifact(filePath = '') {
  const base = path.basename(filePath);
  if (base.endsWith('.done') && !base.startsWith('.')) return false;
  if (base.startsWith('relatorio-') && base.endsWith('.txt')) return false;
  return [/^terminal-.*\.log$/i, /\.lock$/i, /monitor-state\.json$/i].some(r => r.test(base));
}
```

Diferenças com a versão atualizada

Padrão	taskExecutionService	evidenceService
<code>terminal-*.log</code>	Detecta	Detecta
<code>relatorio-*.txt</code>	NÃO é efêmero	É efêmero
<code>.done</code> ocultos	Não detecta	Detecta
<code>.lock</code>	Detecta	Detecta
<code>monitor-state.json</code>	Detecta	Detecta

Impacto

A lógica de progresso pode considerar alterações em relatórios como progresso real quando não deveria.

Bug 4: ContractVerificationService não está sendo importado no taskExecutionService

Localização

src/services/taskExecutionService.js , linhas 1-9 (imports)

Problema

O `ContractVerificationService` foi criado durante a modularização mas **nunca foi importado** no `taskExecutionService.js`.

Código Atual (taskExecutionService.js:1-9)

```
const { PrismaClient } = require('@prisma/client');
const fs = require('fs').promises;
const path = require('path');
const { spawn, execSync } = require('child_process');
const CommandExecutor = require('./commandExecutor');
const agentService = require('./agentService');
// FALTA: const ContractVerificationService = require('./contractVerificationService');
// FALTA: const EvidenceService = require('./evidenceService');

const prisma = new PrismaClient();
```

Sugestão de Correção

Adicionar os imports:

```
const ContractVerificationService = require('./contractVerificationService');
const EvidenceService = require('./evidenceService');
```

Bug 5: EvidenceService também não está sendo importado/usado

Localização

src/services/taskExecutionService.js

Problema

Similar ao bug 4, o `EvidenceService` também foi criado mas não está sendo usado. O código usa a função local `applyExecutionEvidence` em vez de `EvidenceService.applyExecutionEvidence`.

Chamada Problemática (linha 484)

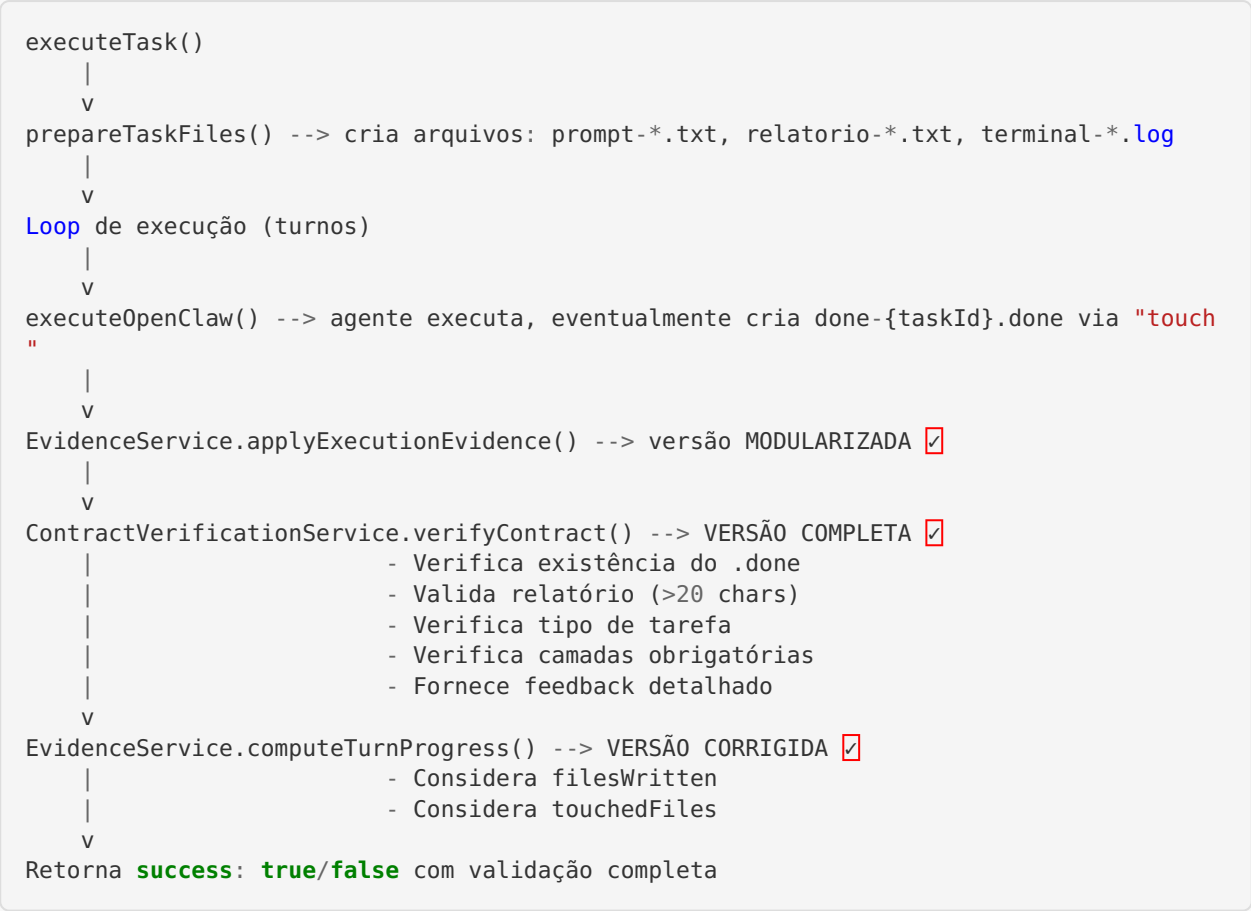
```
this.applyExecutionEvidence(evidence, res.toolCall?.name, res.toolResult || {}, { executionDirectory: project?.pastaBase || TASKS_DIR });
```

Fluxograma: Fluxo Atual vs Esperado

Fluxo ATUAL (com bugs)

```
executeTask()
|
v
prepareTaskFiles() --> cria arquivos: prompt-*.txt, relatorio-*.txt, terminal-*.log
|                      (doneFile definido como done-{taskId}.done mas NÃO criado ainda)
a)
  v
  Loop de execução (turnos)
  |
  v
  executeOpenClaw() --> agente executa, eventualmente cria done-{taskId}.done via "touch"
  |
  v
  this.applyExecutionEvidence() --> versão LOCAL (desatualizada)
  |
  v
  this.verifyContract() --> VERSÃO LOCAL SIMPLIFICADA ❌
  |                      - Apenas verifica se .done existe
  |                      - NÃO valida relatório completo
  |                      - NÃO verifica camadas obrigatórias
  v
  computeTurnProgress() --> VERSÃO LOCAL (desatualizada) ❌
  |                      - NÃO considera filesWritten
  |                      - NÃO considera touchedFiles
  v
  Retorna success: true/false
```

Fluxo ESPERADO (corrigido)



Mapeamento do Fluxo do .done

Onde o .done é DEFINIDO (caminho)

Arquivo	Linha	Código
taskFileService.js	21	doneFile: path.join(tasksDir, \ done-\${taskId}.done`)
taskExecutionService.js	387	const doneFile = path.join(tasksDir, \ done-\${taskId}.done`)

Onde o .done é INSTRUÍDO A SER CRIADO (no prompt do agente)

Arquivo	Linha	Instrução
taskFileService.js	141	2. Use "exec" com 'touch \${paths.doneFile}'
taskExecutionService.js	401	2. Use: {"name": "exec", "arguments": {"command": "touch \${doneFile}"}}

Onde o .done é VERIFICADO

Arquivo	Linha	Método
taskExecutionService.js	446	const doneExists = await this.fileExists(doneFile)
contractVerificationService.js	21	const doneExists = await fs.access(doneFile).then(..)

O que acontece quando .done é DETECTADO

Versão local (taskExecutionService.js:456):

```
if (doneExists) return { contractFulfilled: true, executionNotes: executionNotes || 'Concluído.' };
```

- Retorna imediatamente como sucesso, SEM validar relatório ou camadas

Versão completa (contractVerificationService.js):

- Verifica tipo de tarefa
- Se análise: exige relatório válido + evidência de inspeção
- Se desenvolvimento: exige relatório + arquivos modificados nas camadas obrigatórias
- Se automação: exige evidência de execução útil

O que acontece quando .done NÃO é detectado

- Continua o loop de execução
 - Incrementa contador de turnos sem progresso
 - Se turnosSemProgresso >= 6 , marca como estagnação e falha
-

Resumo dos Bugs por Severidade

#	Bug	Severidade	Arquivo	Linha
1	<code>verifyContract</code> duplicado, usando versão simplificada	CRÍTICO	<code>taskExecution-Service.js</code>	444-458, 485
2	<code>computeTurnProgress</code> duplicado, sem considerar <code>filesWritten/touchedFiles</code>	CRÍTICO	<code>taskExecution-Service.js</code>	551-563, 498
3	<code>isEphemeralArtifact</code> duplicado com lógica diferente	MODERADO	<code>taskExecution-Service.js</code>	515-519
4	<code>ContractVerificationService</code> não importado	CRÍTICO	<code>taskExecution-Service.js</code>	imports
5	<code>EvidenceService</code> não importado	CRÍTICO	<code>taskExecution-Service.js</code>	imports

Recomendações (SEM IMPLEMENTAR)

1. **URGENTE:** Importar `ContractVerificationService` e `EvidenceService` no `taskExecutionService.js`
2. **URGENTE:** Substituir chamadas de `this.verifyContract()` por `ContractVerificationService.verifyContract()`
3. **URGENTE:** Substituir chamadas de `computeTurnProgress()` local por `EvidenceService.computeTurnProgress()`
4. **URGENTE:** Substituir chamadas de `this.applyExecutionEvidence()` por `EvidenceService.applyExecutionEvidence()`
5. **MODERADO:** Remover as funções duplicadas locais do `taskExecutionService.js` após migração
6. **MODERADO:** Adicionar testes de integração para validar o fluxo completo

Conclusão

O principal problema é que a **modularização foi feita parcialmente**: os serviços foram criados (`ContractVerificationService` , `EvidenceService`) com lógica atualizada e corrigida, mas o `taskExecutionService.js` **ainda mantém cópias locais desatualizadas** e está usando-as. Isso causa:

1. Falsos-positivos: tarefas marcadas como concluídas sem validação completa
2. Falsos-negativos: tarefas de análise marcadas como “sem progresso” quando há evidência de leitura
3. Feedback inadequado: agente não recebe instruções detalhadas de correção

A correção é relativamente simples: importar os serviços modularizados e usar suas implementações no lugar das cópias locais.